

Presentación Ejecutiva: Proyecto Ecommify

1. Contexto

Ecommify es una plataforma de e-commerce que gestiona el ciclo de vida completo de los pedidos. Se basa en el dataset de Olist para modelar un sistema híbrido realista, separando las entidades transaccionales críticas de los datos variables y enriquecidos del catálogo para optimizar la escalabilidad.

2. Requisitos Principales

- **Funcionales:** Registro de clientes, gestión de pedidos multitem e integración de pagos, envíos y reseñas.
- **No Funcionales:** Integridad referencial, disponibilidad alta, trazabilidad y soporte para esquemas flexibles en el catálogo.

3. Diagrama ER y Entidades

El modelo conceptual vincula clientes con pedidos, y estos con ítems que referencian productos y vendedores. Las restricciones de negocio aseguran que no existan ítems sin pedido o entregas sin datos de envío, garantizando la consistencia del flujo operativo.

4. Diseño Lógico: Esquema Relacional (PostgreSQL)

PostgreSQL actúa como el núcleo transaccional. El esquema está normalizado en 3FN para las tablas core: **customers**, **orders**, **order_items**, **payments**, **sellers** y **shipments**. Utiliza UUIDs como llaves primarias para facilitar la distribución de datos.

5. Tipos Avanzados y Extensiones

Se utiliza **JSONB** para snapshots de atributos en el momento de la compra y **TIMESTAMPTZ** para precisión temporal. Se integran extensiones como *pg_trgm* para búsquedas semánticas y *uuid-oss* para la gestión de identificadores únicos.

6. Diseño Lógico: Esquema MongoDB

MongoDB gestiona el catálogo enriquecido mediante colecciones como *catalog_products* y *product_content*. Aplica patrones de **Embedding** para metadatos SEO y **Referencing** para vincular eventos de comportamiento del usuario.

7. Matriz de Decisión Tecnológica

- **PostgreSQL:** Elegido para pedidos, pagos y clientes debido a su soporte ACID y cumplimiento de integridad fuerte.
- **MongoDB:** Elegido para atributos de producto y eventos por su flexibilidad de esquema y velocidad en lecturas de documentos complejos.

8. Análisis CAP y Arquitectura Híbrida

El sistema equilibra la Consistencia (C) en el módulo de pagos mediante PostgreSQL, priorizando la Disponibilidad (A) y Tolerancia a Particiones (P) en el módulo de catálogo con MongoDB. La arquitectura separa responsabilidades: lo transaccional es rígido y seguro; lo informacional es fluido y escalable.

9. Limitaciones y Mitigaciones

La principal limitación es la complejidad de mantener la consistencia eventual entre ambos motores. Se mitiga mediante el uso de snapshots en PostgreSQL para datos de producto y procesos de sincronización asíncrona para el catálogo.

10. Plan de Implementación

Fase 1: Despliegue de esquema relacional base en PostgreSQL. Fase 2: Migración del catálogo y contenido enriquecido a MongoDB. Fase 3: Integración de servicios de orquestación para la arquitectura híbrida.